



Batch Processing II

(v1.00)

Week 5: October 6, 2026

Jimmy Lin
David R. Cheriton School of Computer Science
University of Waterloo

These slides are available at <https://lintool.github.io/cs431-2026f/>

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International
See <https://creativecommons.org/licenses/by-nc-sa/4.0/> for details



Key Questions

In what ways does Spark improve over MapReduce?

How is distributed group by implemented efficiently at scale?

How do commutative and associative operations contribute to efficient distributed execution?

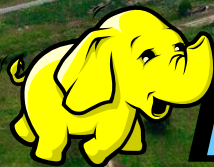
How does partitioning contribute to efficient distributed execution?

How do these concepts come together in efficient joins at scale?

The datacenter *is* the computer!

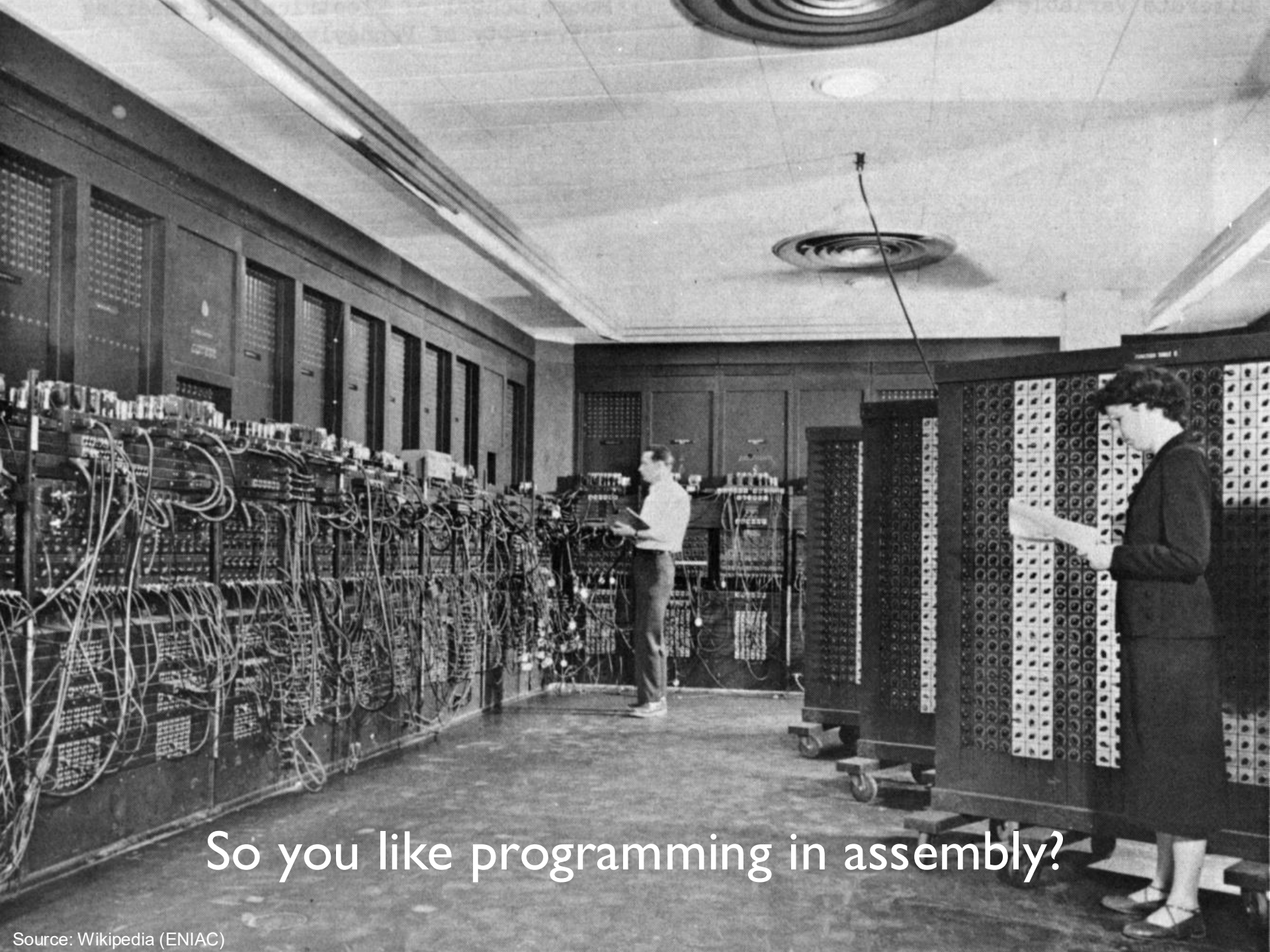
What's the instruction set?

map / reduce



hadoop





So you like programming in assembly?



The datacenter *is* the computer!

What's the instruction set?

map / filter / flatMap ...
groupByKey / reduceByKey / join / cogroup ...

That's it.

MapReduce

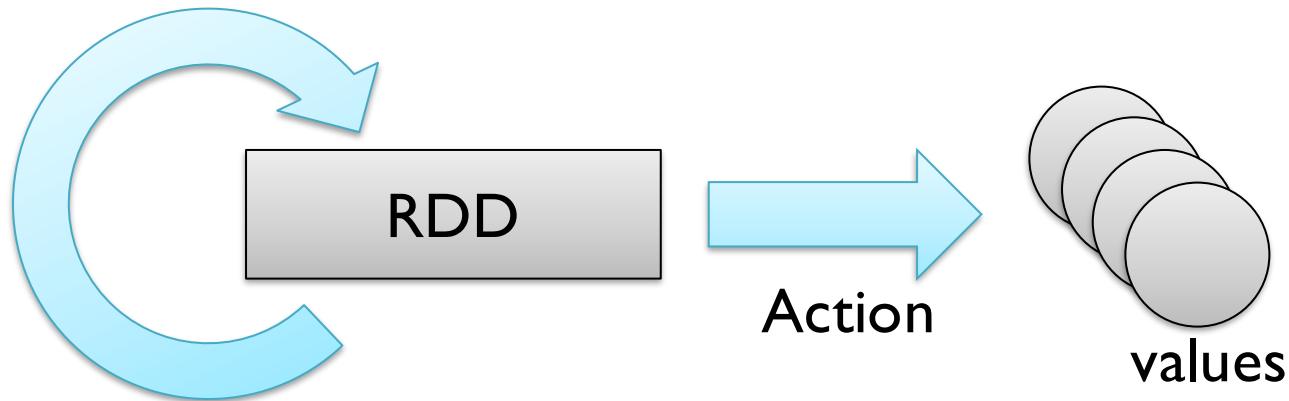
```
results1 = records.map(...)  
                .reduce(...)  
  
results2 = results1.map(...)  
                .reduce(...)
```

Spark

```
results = rdd.foo(...)  
                .bar(...)  
                .baz(...)
```

RDD Lifecycle

Transformation

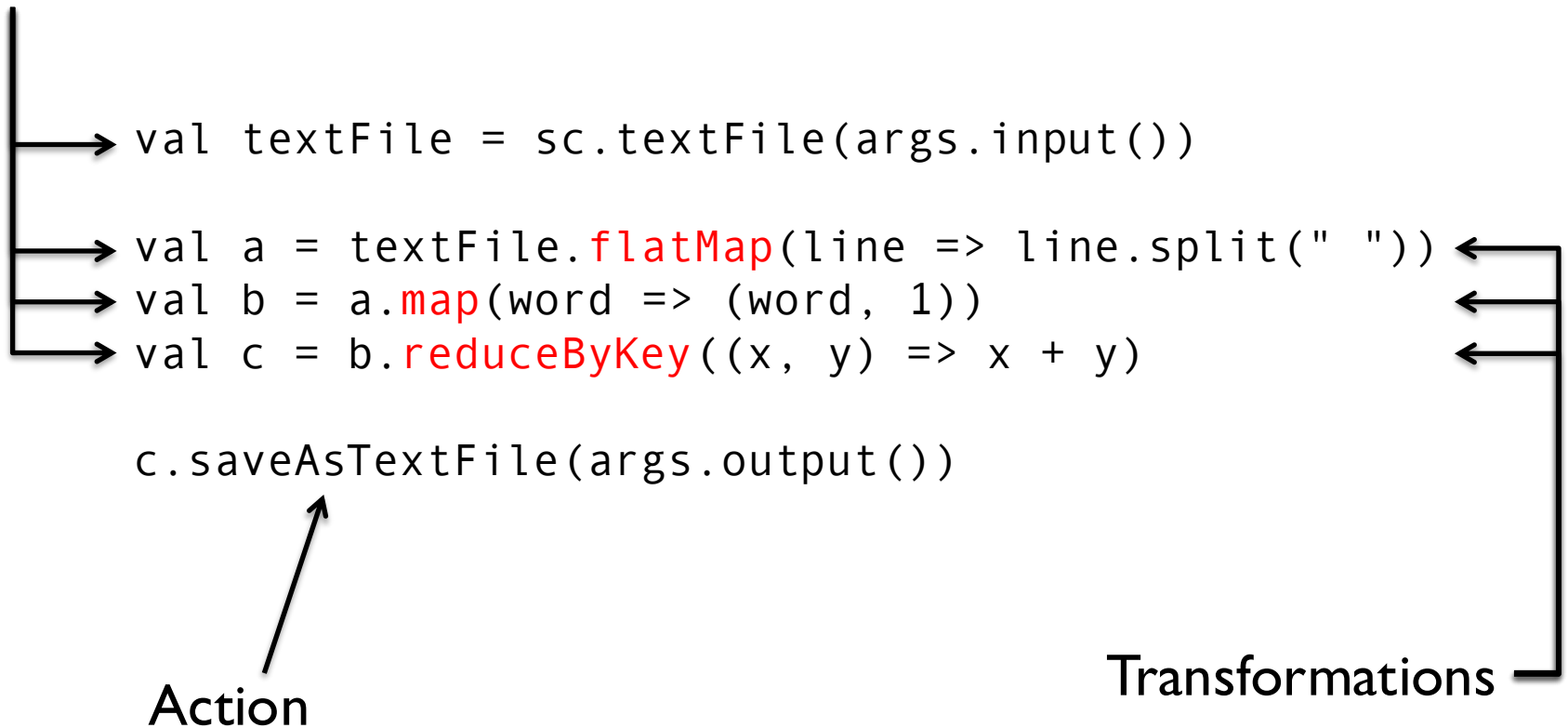


Transformations are lazy:
Framework keeps track of lineage

Actions trigger actual execution

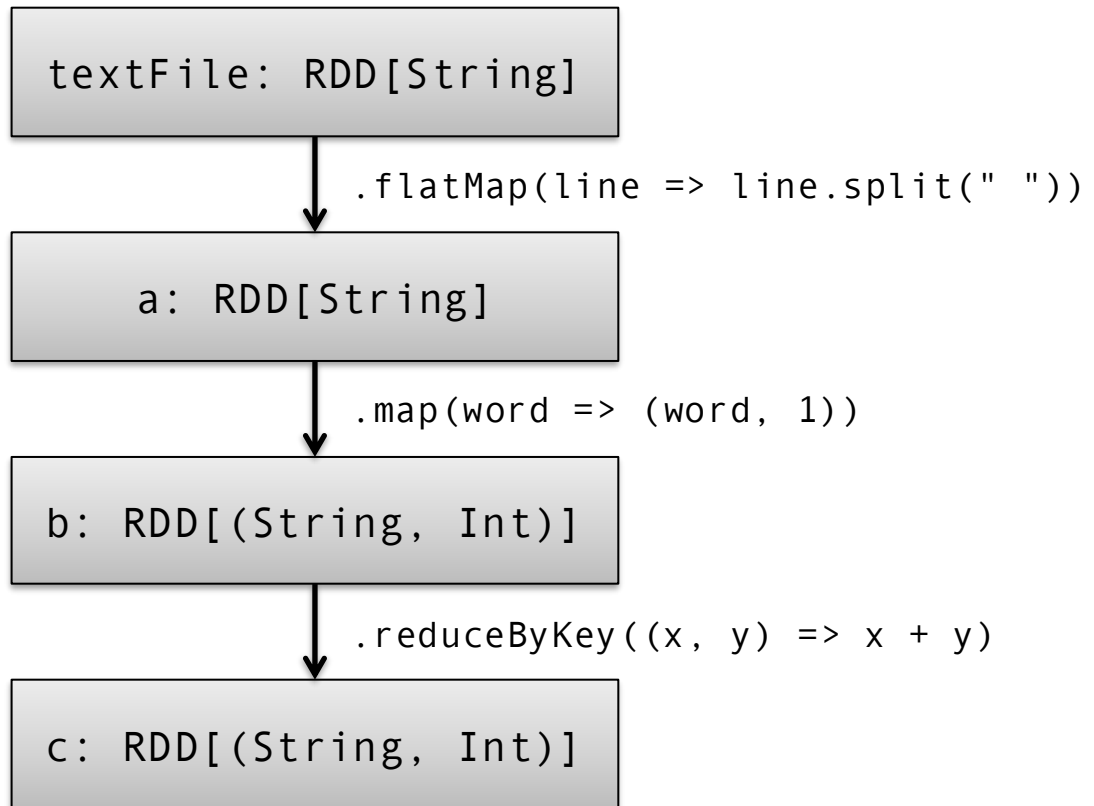
Spark Word Count

RDDs



RDDs and Lineage

On HDFS



Action!

**Remember,
transformations are lazy!**

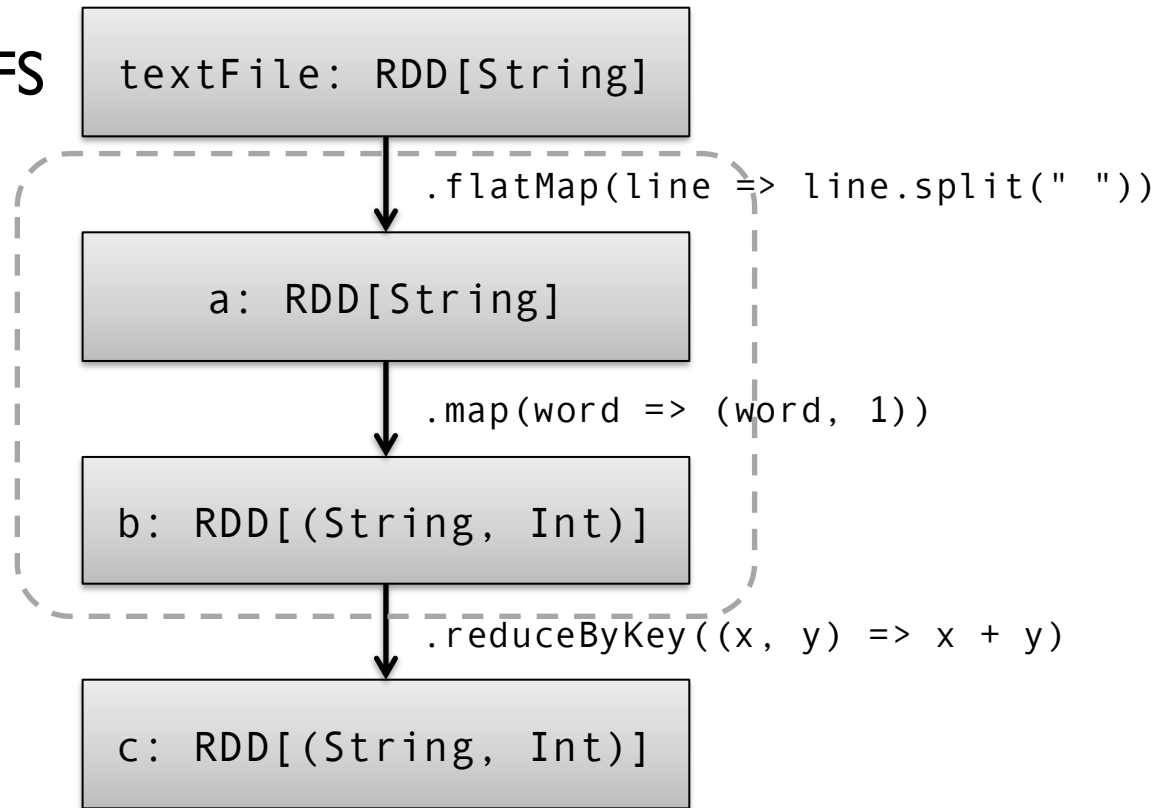
RDDs and Optimizations

Lazy evaluation creates optimization opportunities

On HDFS

RDDs don't need
to be materialized!

Want MM?

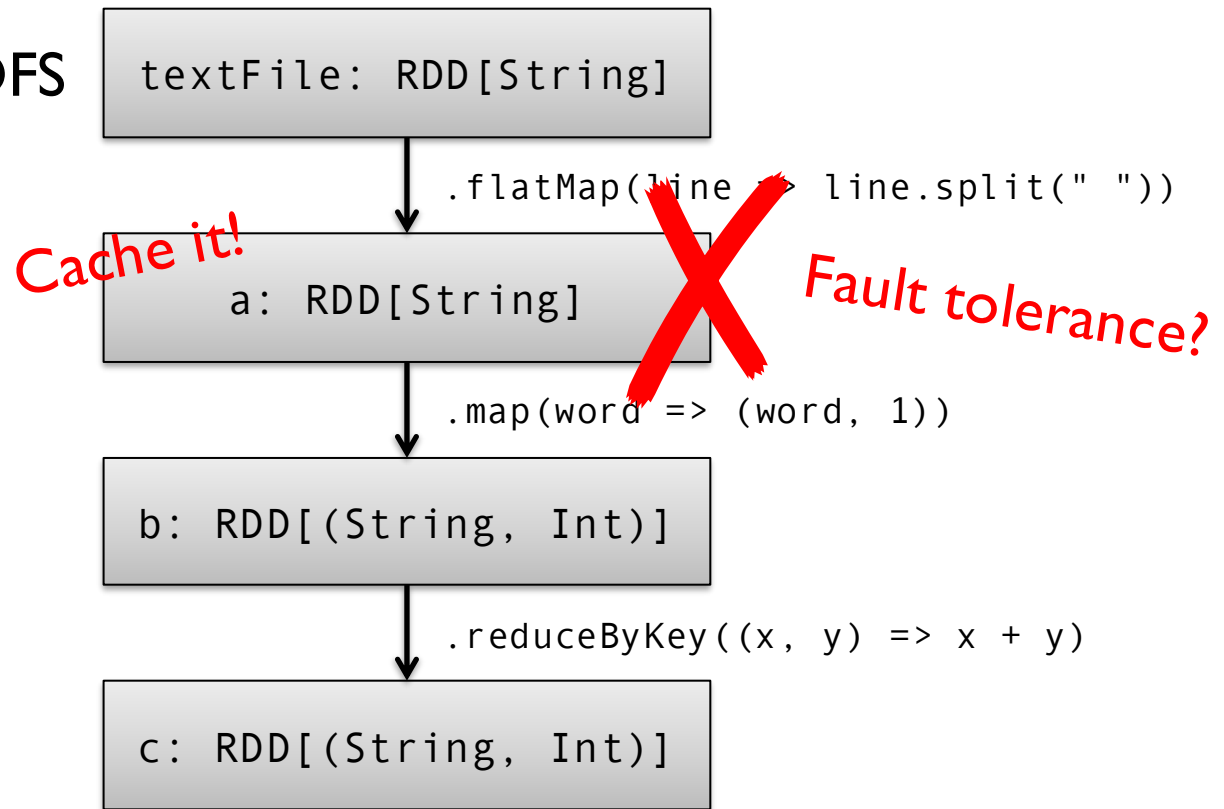


Action!

RDDs and Caching

RDDs can be materialized in memory (and on disk)!

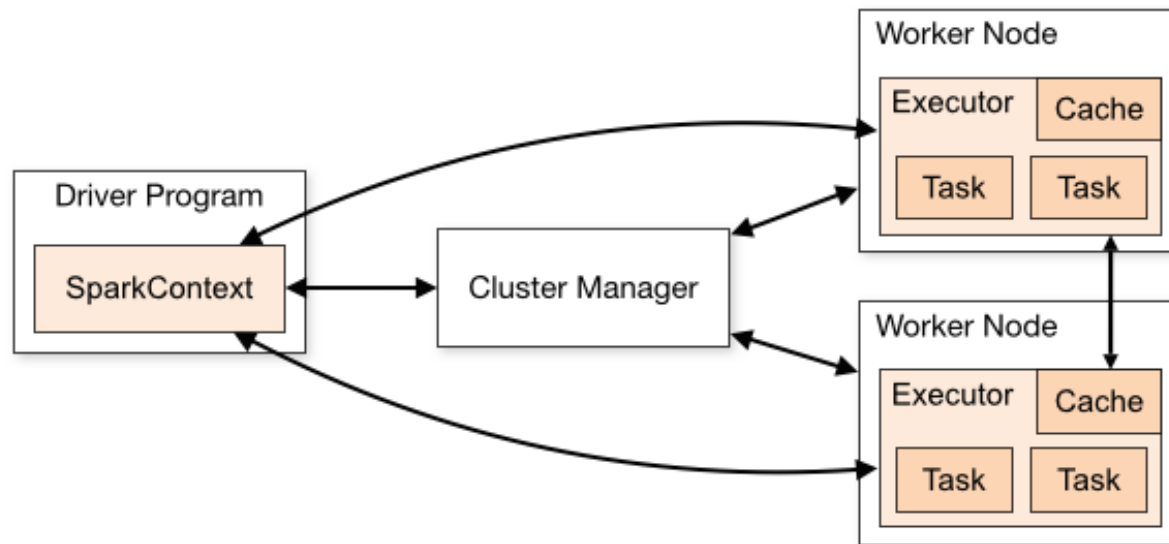
On HDFS



Action!

Spark works even if the RDDs are *partially* cached!

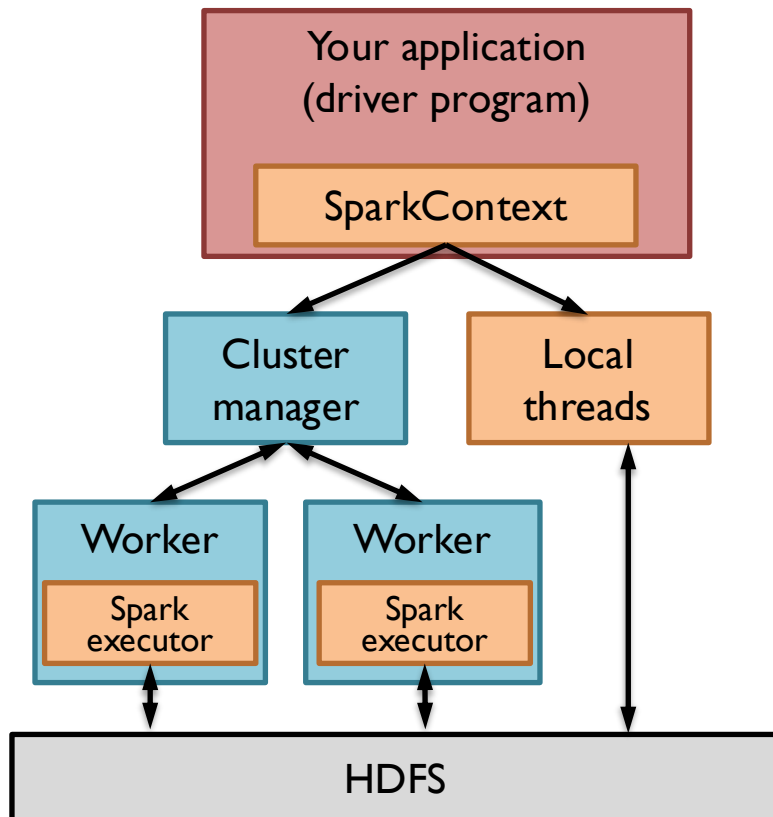
Spark Cluster



Spark Programs

Scala, Java, Python, R

spark-shell spark-submit



Spark context: tells the framework where to find the cluster

Use it to create RDDs to get started...

```
val textFile =
  sc.textFile(args.input())

textFile
  .flatMap(line => tokenize(line))
  .map(word => (word, 1))
  .reduceByKey((x, y) => x + y)
  .saveAsTextFile(args.output())
```

What happens to the functions?

Where do the results go?

Beware of `collect()`!

Technical Deep Dives

Implementation of distributed group by

... implications for algorithm design

The importance of partitioning

Case study in relational joins

More? Why do you care?

The essence of abstraction is preserving information that is relevant in a given context, and forgetting information that is irrelevant in that context.

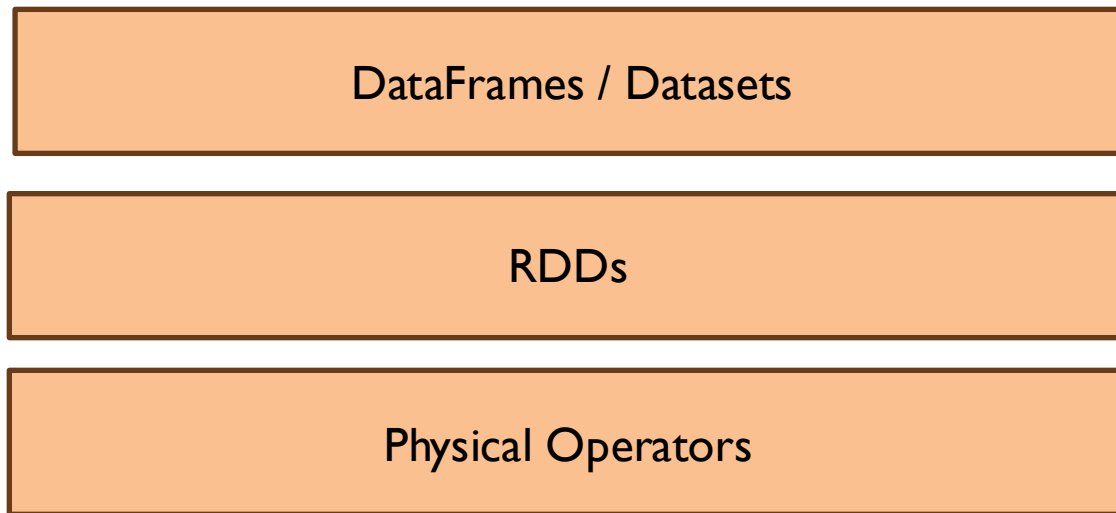
– computer scientist John V. Guttag

1. All abstractions are *leaky*
2. Important to develop intuitions
3. What do you want to be?
4. Curiosity

You don't have to be an engineer to be a racing driver,
but you do have to have mechanical sympathy

– Formula One driver Jackie Stewart

Spark Stack



Who lives where?

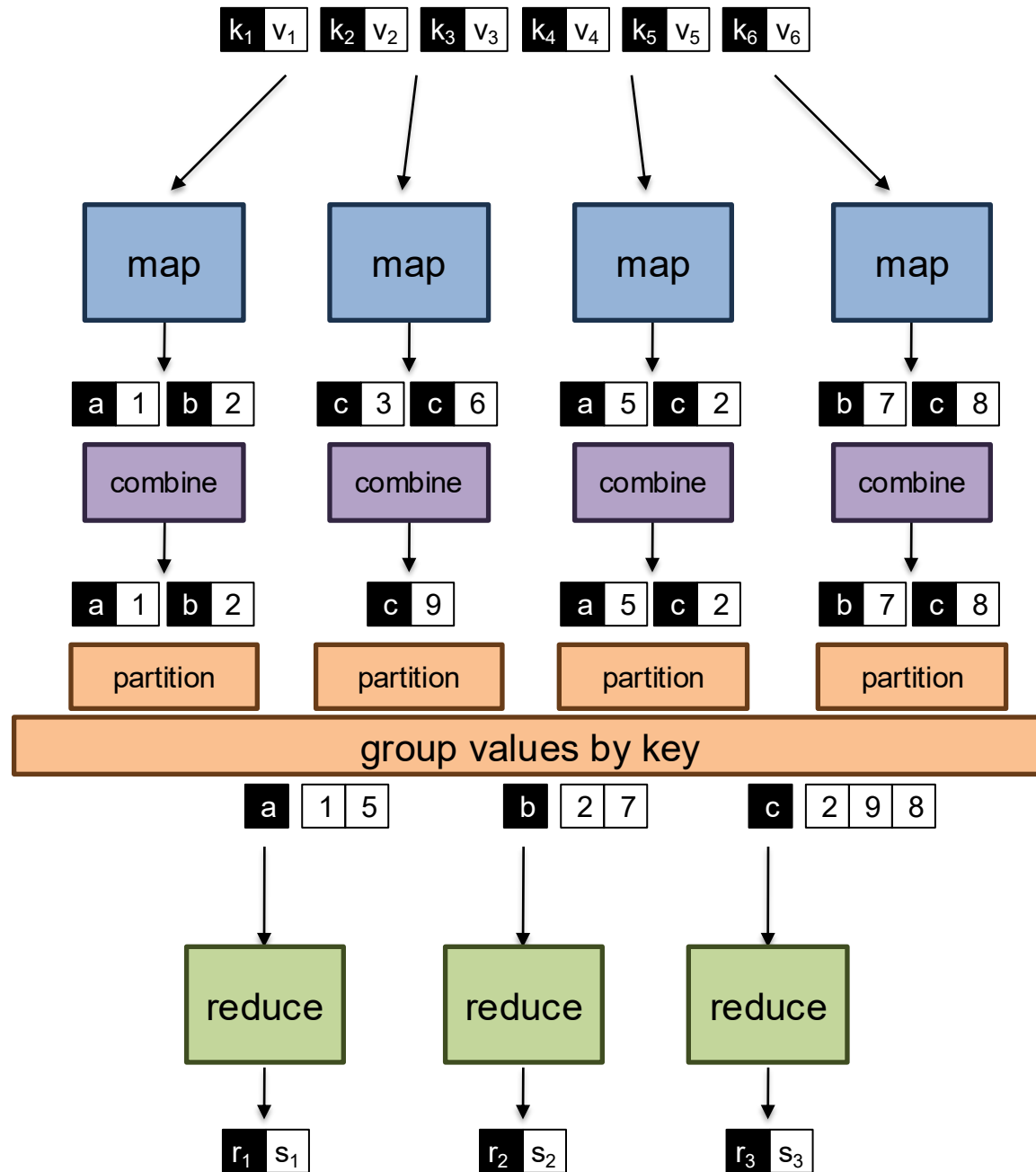
Technical Deep Dives

Implementation of distributed group by

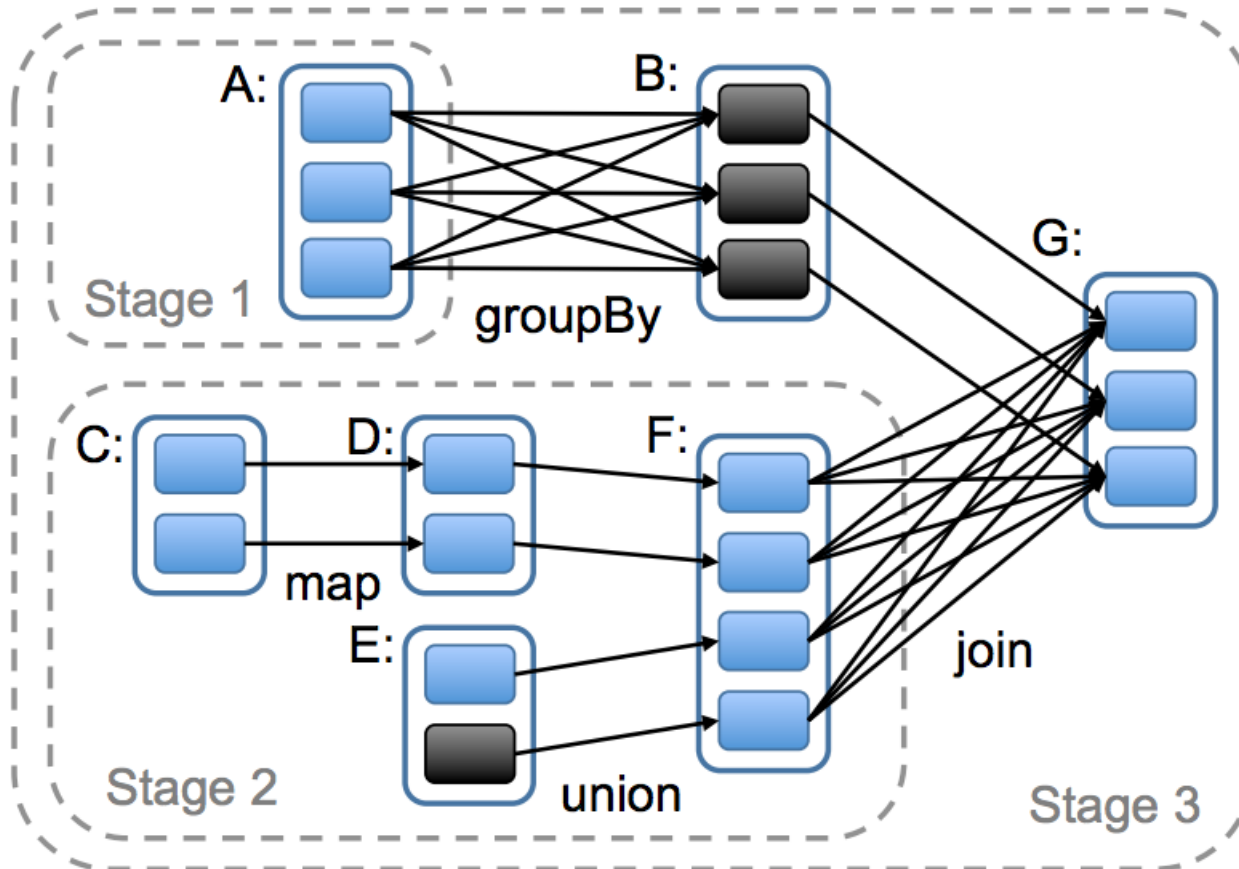
... implications for algorithm design

The importance of partitioning

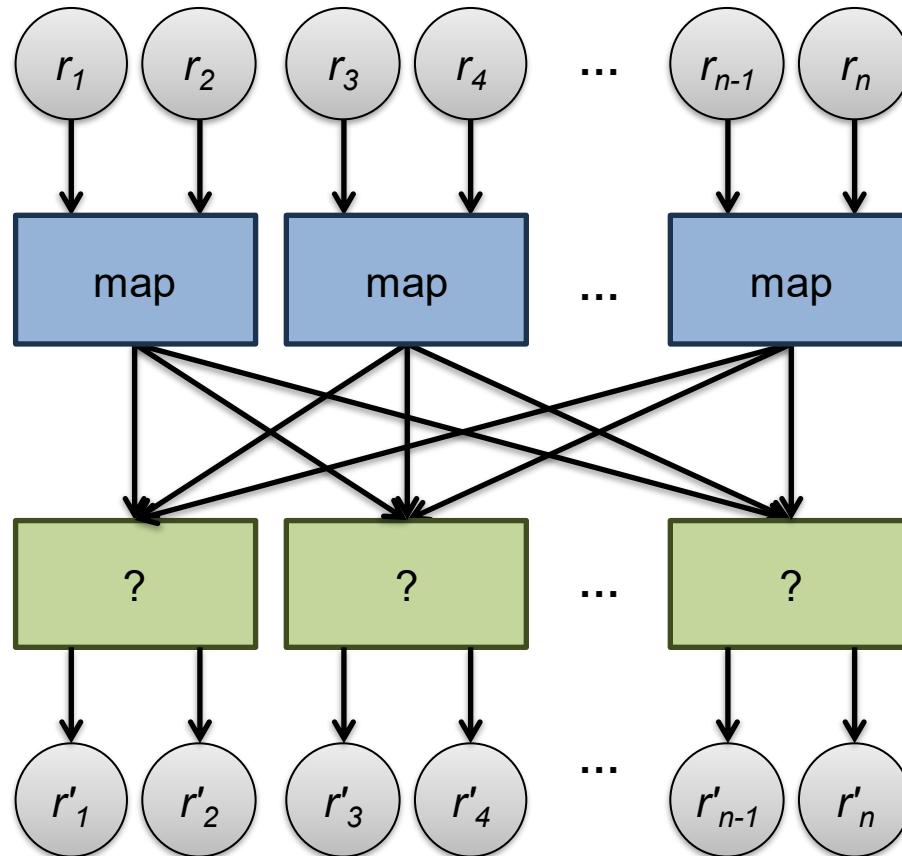
Case study in relational joins



Spark Execution Plan



We need communication



How is this implemented?

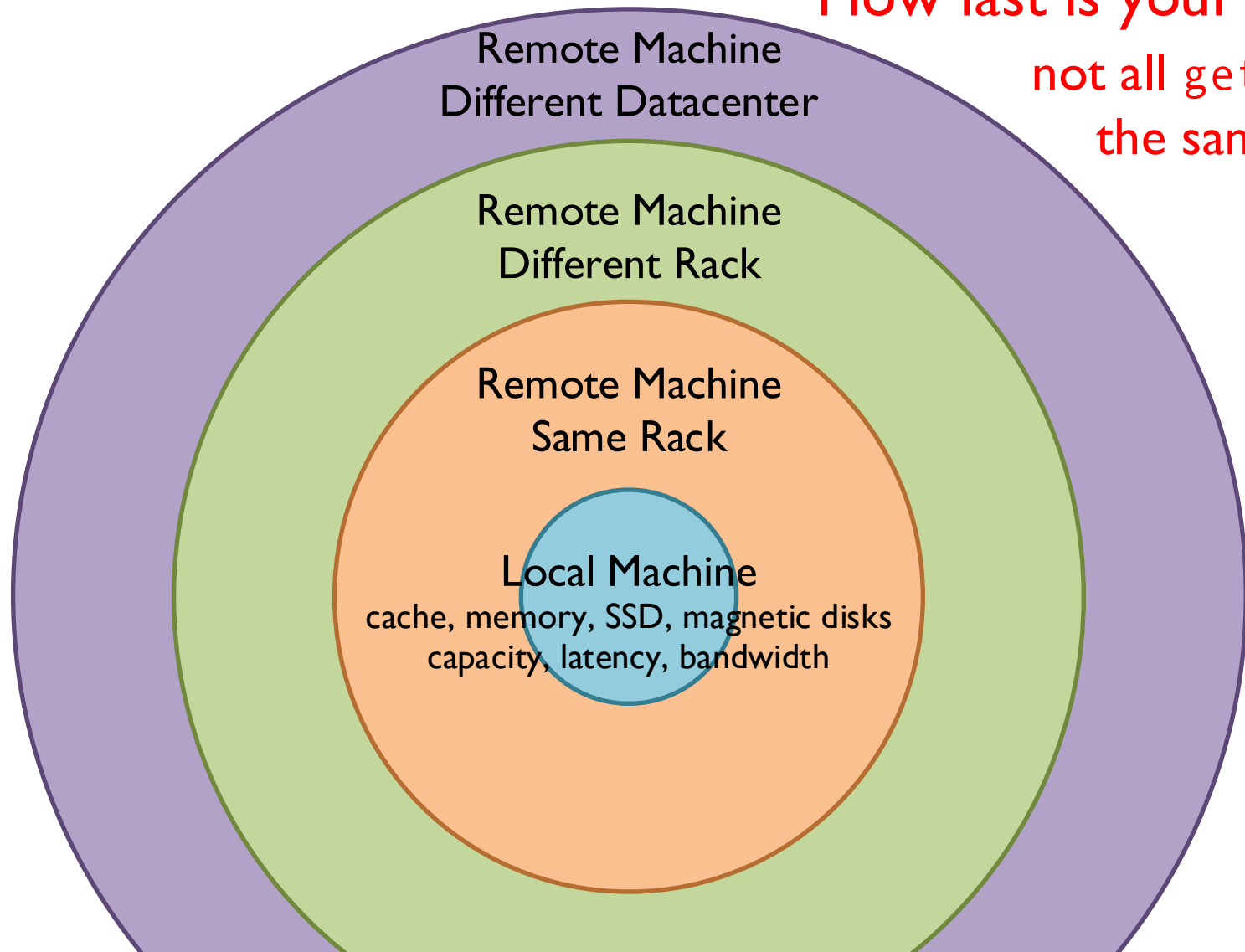
Why do you care?

1. All abstractions are *leaky*
2. Important to develop intuitions
3. What do you want to be?
4. Curiosity

All abstractions are *leaky*
example: `get(x)`

Storage Hierarchy

How fast is your get?
not all gets are
the same!



All abstractions are *leaky*
example: `get(x)`

You shouldn't need to care about locality, but you kinda need to...

Here's another one...

Seek vs. Scans

Consider a 1 TB database with 100-byte records

We want to update 1 percent of the records

Scenario 1: Mutate each record

Each update takes ~30 ms (seek, read, write)

10^8 updates = ~35 days

Scenario 2: Rewrite all records

Assume 100 MB/s throughput

Time = 5.6 hours(!)

Lesson? Random access is expensive!

What's the point?

At scale, sorting is the most efficient implementation
of distributed group by!

Sort/Shuffle/Merge MapReduce

Map side

Map outputs are buffered in memory in a circular buffer

When buffer reaches threshold, contents are “spilled” to disk

Spills are merged into a single, partitioned file (sorted within each partition)

Combiner runs during the merges

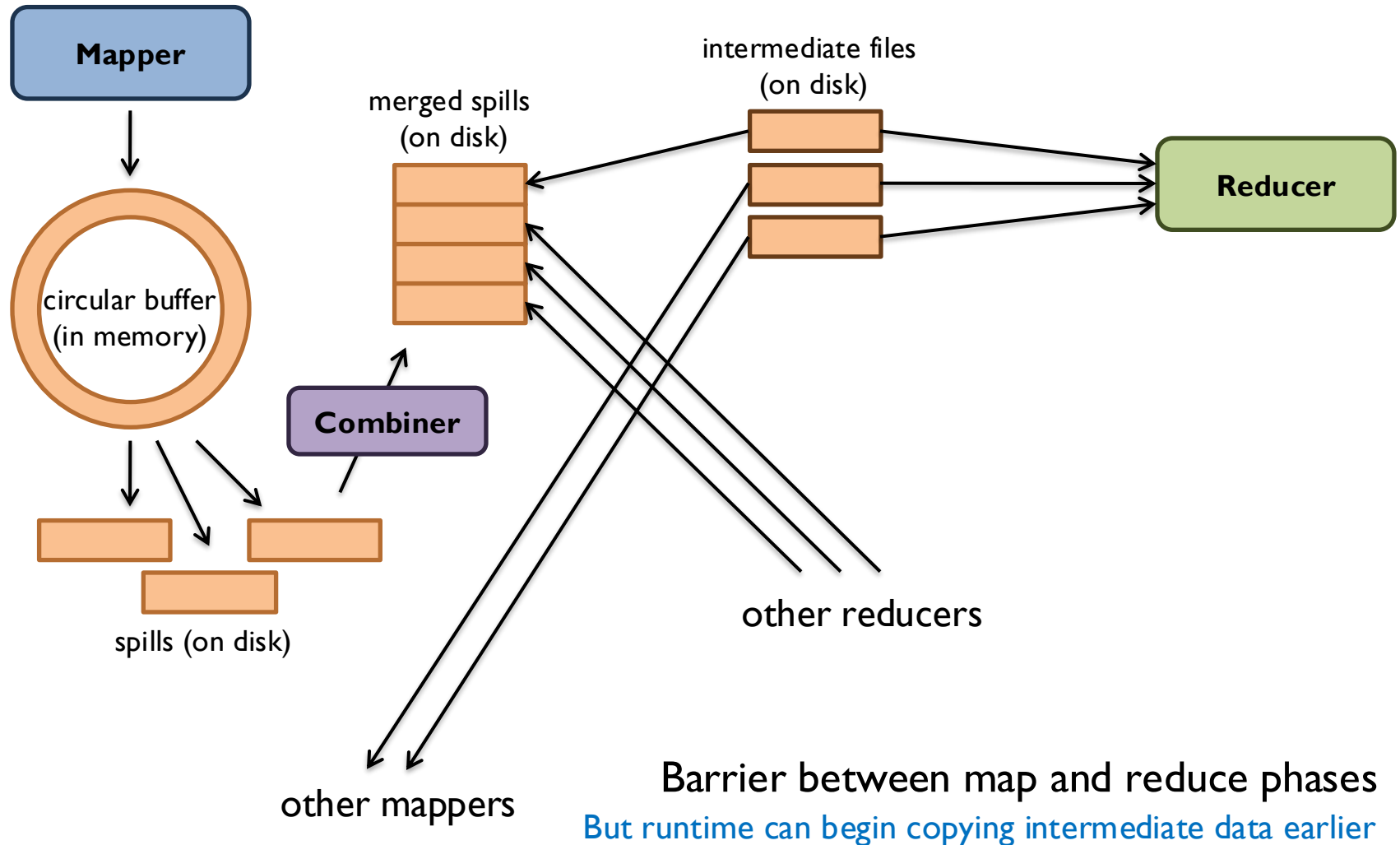
Reduce side

Map outputs are copied over to the reducer machines

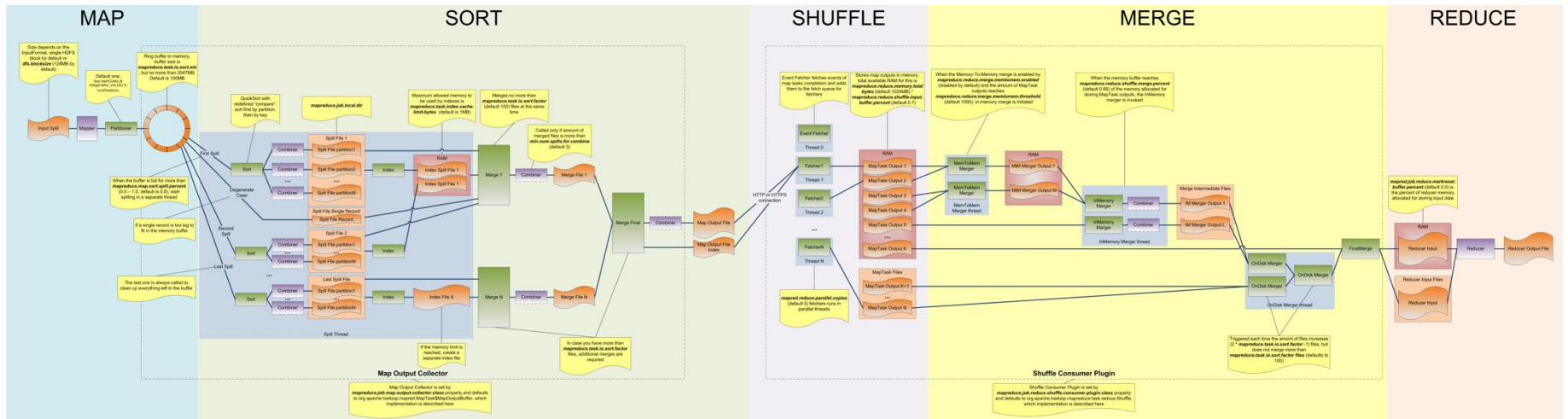
“Sort” is a multi-pass merge of map outputs (happens in memory and on disk)

Final merge pass goes directly into reducer

Sort/Shuffle/Merge MapReduce

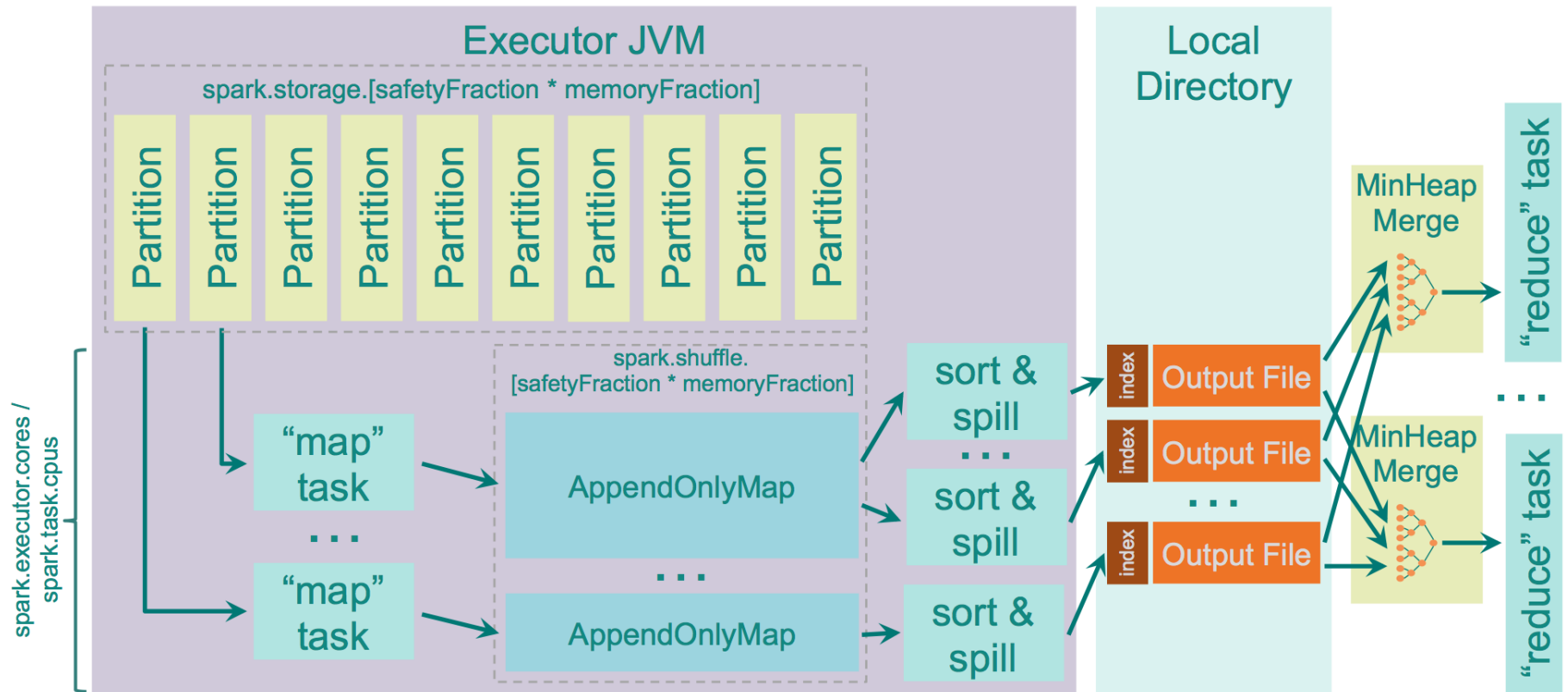


Sort/Shuffle/Merge MapReduce



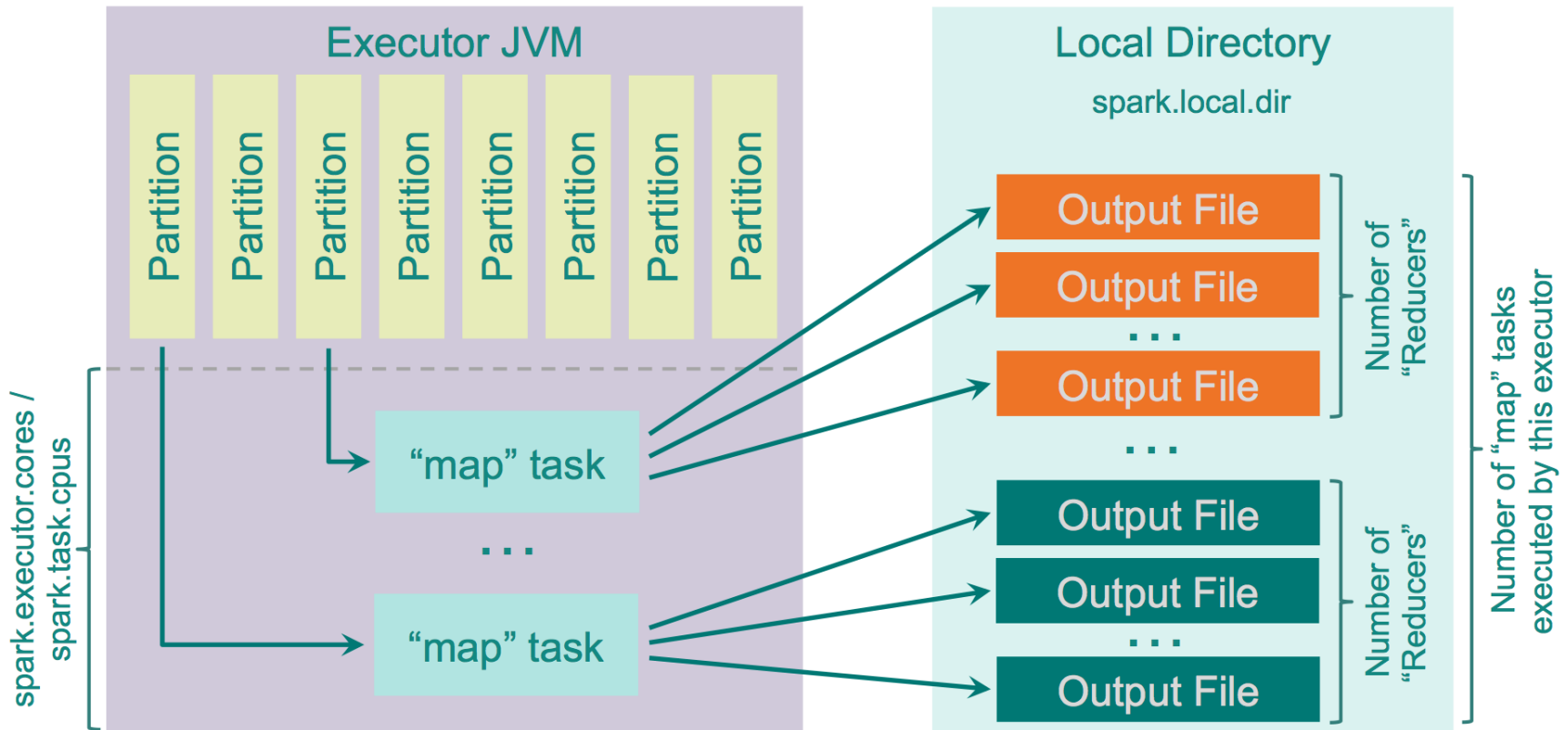
Spark Shuffle Implementations

Sort shuffle



Spark Shuffle Implementations

Hash shuffle



What's the point?

Technical Deep Dives

Implementation of distributed group by

... implications for algorithm design

The importance of partitioning

Case study in relational joins

Super powers:
Associativity and Commutativity!

The Power of Associativity

You can put parentheses wherever you want!

$$\left(v_1 \oplus v_2 \oplus v_3 \right) \oplus \left(v_4 \oplus v_5 \oplus v_6 \oplus v_7 \right) \oplus \left(v_8 \oplus v_9 \right)$$

$$\left(v_1 \oplus v_2 \right) \oplus \left(v_3 \oplus v_4 \oplus v_5 \right) \oplus \left(v_6 \oplus v_7 \oplus v_8 \oplus v_9 \right)$$

$$\left(v_1 \oplus v_2 \oplus \left(v_3 \oplus v_4 \oplus v_5 \right) \right) \oplus \left(v_6 \oplus v_7 \oplus v_8 \oplus v_9 \right)$$

The Power of Commutativity

You can swap the order of operands however you want!

$$\left[v_1 \oplus v_2 \oplus v_3 \right] \oplus \left[v_4 \oplus v_5 \oplus v_6 \oplus v_7 \right] \oplus \left[v_8 \oplus v_9 \right]$$

$$\left[v_4 \oplus v_5 \oplus v_6 \oplus v_7 \right] \oplus \left[v_1 \oplus v_2 \oplus v_3 \right] \oplus \left[v_8 \oplus v_9 \right]$$

$$\left[v_8 \oplus v_9 \right] \oplus \left[v_4 \oplus v_5 \oplus v_6 \oplus v_7 \right] \oplus \left[v_1 \oplus v_2 \oplus v_3 \right]$$

Implications for distributed processing?

You don't know when the tasks begin

You don't know when the tasks end

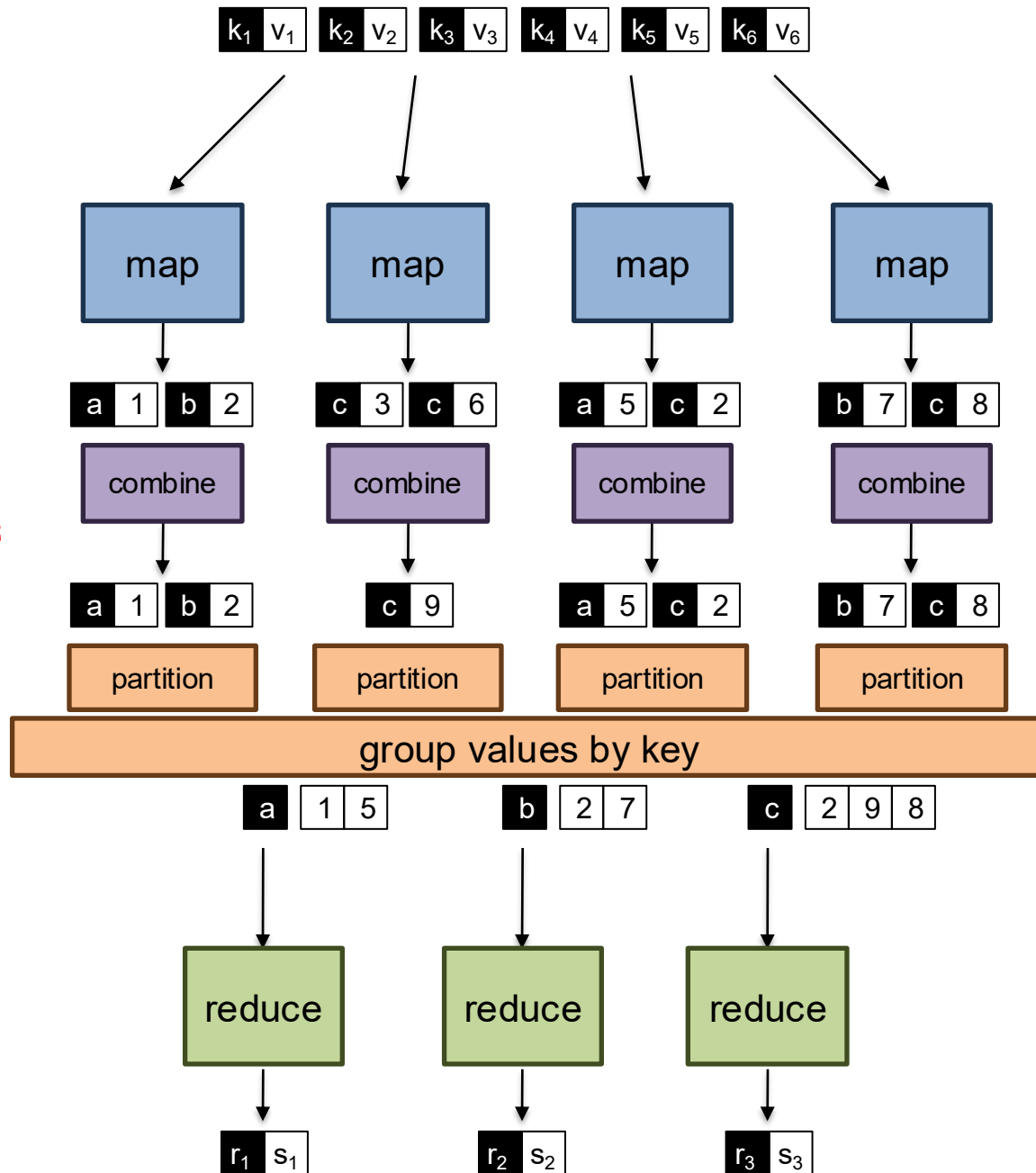
You don't know when the tasks interrupt each other

You don't know when intermediate data arrive

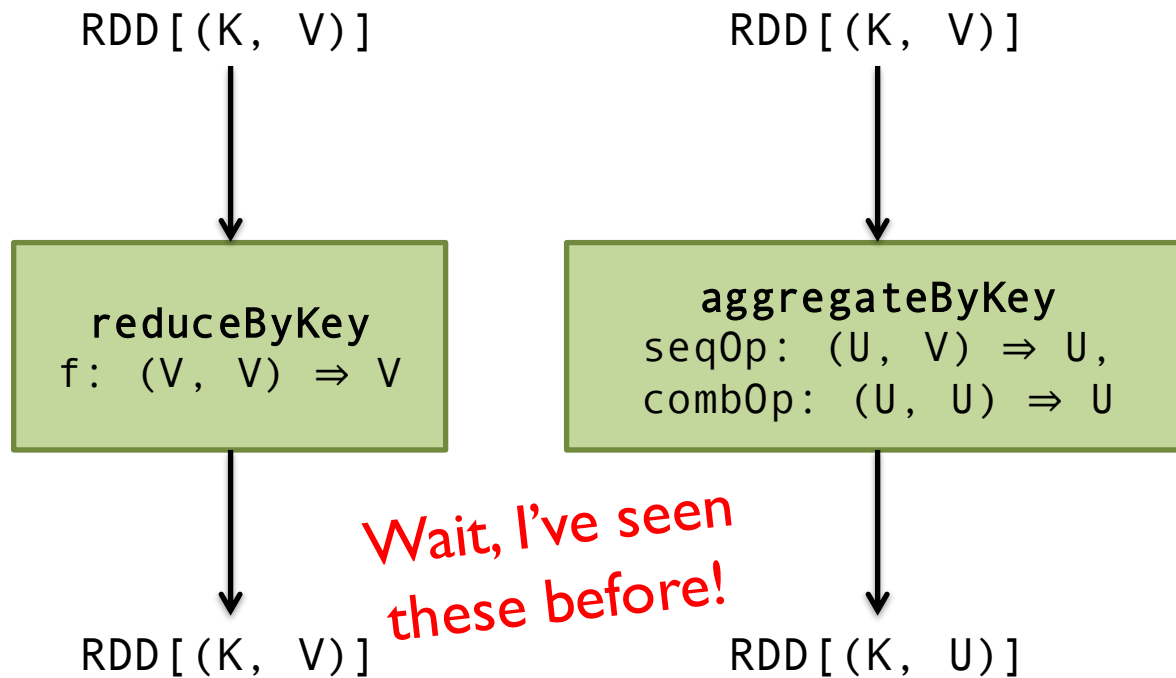
...

It's okay!

When can
reducers be
combiners?



Back to these...

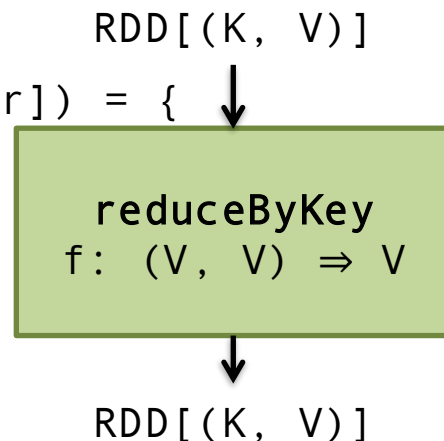


Computing the Mean: Version I

```
class Mapper {  
  def map(key: String, value: Int) = {  
    emit(key, value)  
  }  
}  
  
class Reducer {  
  def reduce(key: String, values: Iterable[Int]) {  
    for (value <- values) {  
      sum += value  
      cnt += 1  
    }  
    emit(key, sum/cnt)  
  }  
}
```

Computing the Mean: Version 3

```
class Mapper {  
  def map(key: String, value: Int) =  
    context.write(key, (value, 1))  
}  
class Combiner {  
  def reduce(key: String, values: Iterable[Pair]) = {  
    for ((s, c) <- values) {  
      sum += s  
      cnt += c  
    }  
    emit(key, (sum, cnt))  
  }  
}  
class Reducer {  
  def reduce(key: String, values: Iterable[Pair]) = {  
    for ((s, c) <- values) {  
      sum += s  
      cnt += c  
    }  
    emit(key, sum/cnt)  
  }  
}
```



Co-occurrence Matrix: Stripes

```
class Mapper {
  def map(key: Long, value: String) = {
    for (u <- tokenize(value)) {
      val map = new Map()
      for (v <- neighbors(u)) {
        map(v) += 1
      }
      emit(u, map)
    }
  }
}
```

$$\begin{array}{rcl}
 & a \rightarrow \{ b: 1, c: 2, d: 5, e: 3, f: 2 \} & \\
 & a \rightarrow \{ b: 1, \quad \quad d: 5, e: 3 \} & \\
 + & a \rightarrow \{ b: 1, c: 2, d: 2, \quad \quad f: 2 \} & \\
 \hline
 & a \rightarrow \{ b: 2, c: 2, d: 7, e: 3, f: 2 \} &
 \end{array}$$

```
class Reducer {
  def reduce(key: String, values: Iterable[Map]) = {
    val map = new Map()
    for (value <- values) {
      map += value
    }
    emit(key, map)
  }
}
```

RDD[(K, V)]
 ↓

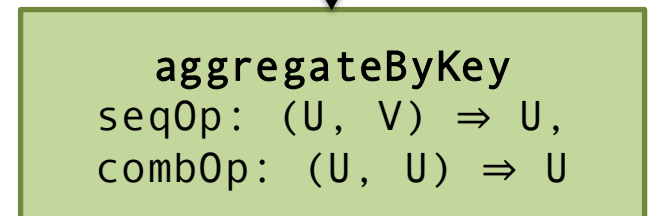
reduceByKey
 f: (V, V) ⇒ V

 ↓
 RDD[(K, V)]

Computing the Mean: Version 2

```
class Mapper {  
  def map(key: String, value: Int) =  
    context.write(key, value)  
}  
class Combiner {  
  def reduce(key: String, values: Iterable[Int]) = {  
    for (value <- values) {  
      sum += value  
      cnt += 1  
    }  
    emit(key, (sum, cnt))  
  }  
}  
class Reducer {  
  def reduce(key: String, values: Iterable[Pair]) = {  
    for ((s, c) <- values) {  
      sum += s  
      cnt += c  
    }  
    emit(key, sum/cnt)  
  }  
}
```

RDD[(K, V)]



RDD[(K, U)]

Super powers:
Associativity and Commutativity!

(But what's the fallback option?)

Technical Deep Dives

Implementation of distributed group by
... implications for algorithm design

The importance of partitioning

Case study in relational joins

Why does partitioning matter?

Ideal Scaling: Why not?

Communication is unavoidable

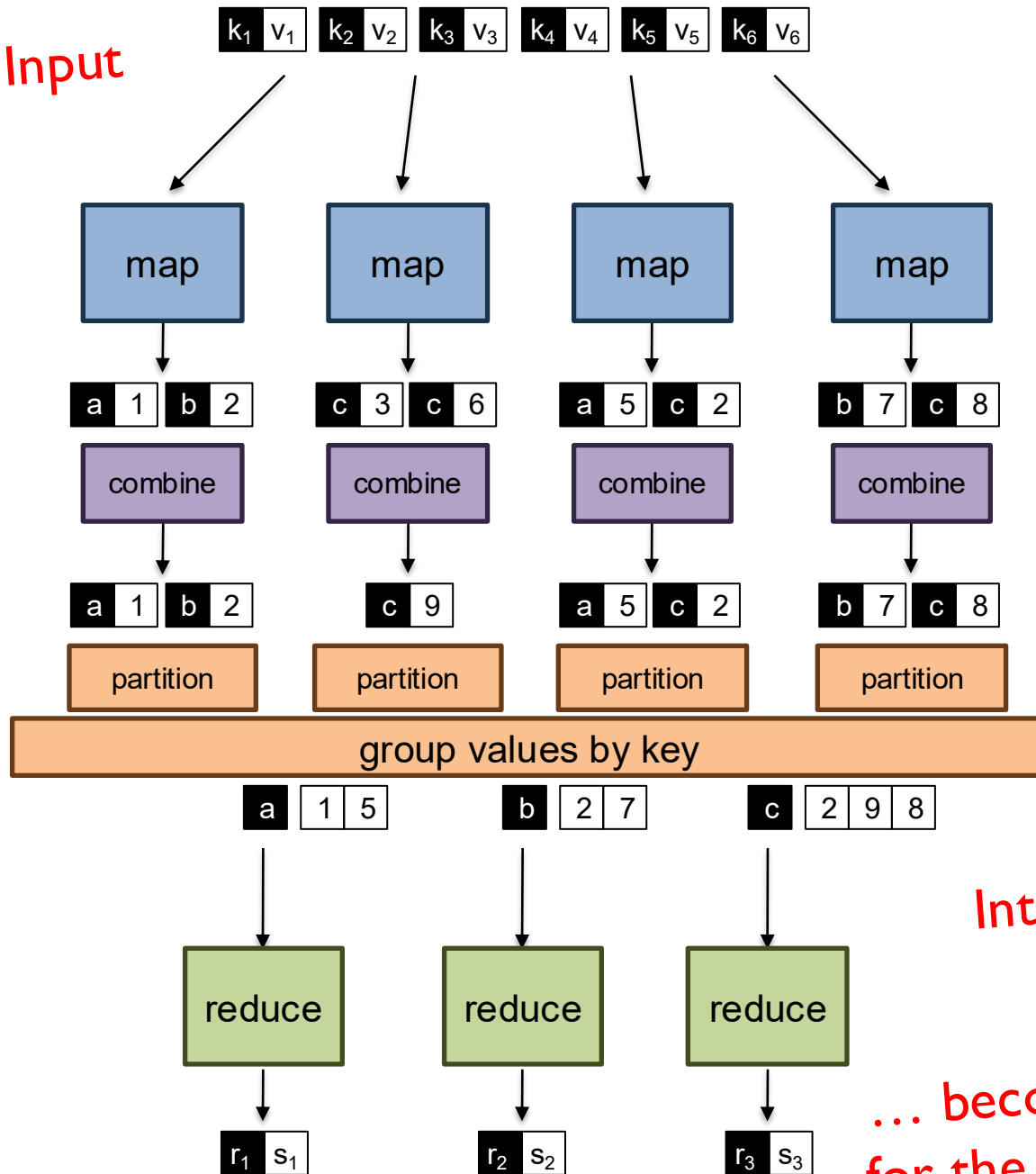
Workers need to share intermediate results...
Which requires communication across machines...
Which requires synchronization...
Which kills performance.

Skew creates idle workers

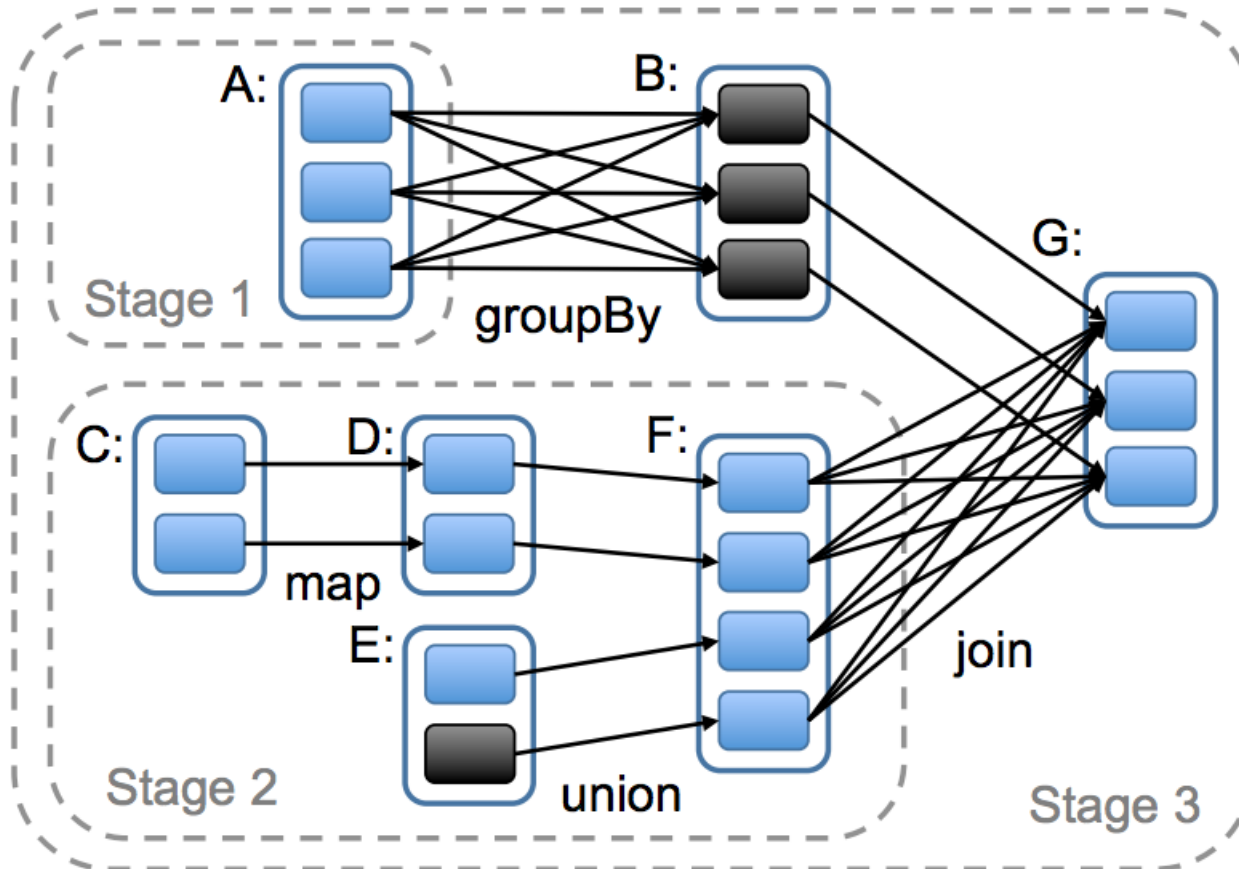
Tasks are never divided perfectly evenly...
And even if they are, processing times can be unpredictable...
Which leads to idle workers.

Why does partitioning matter?
Better partitioning remediates skew!

Skew in Input



Spark Execution Plan



Partition challenges

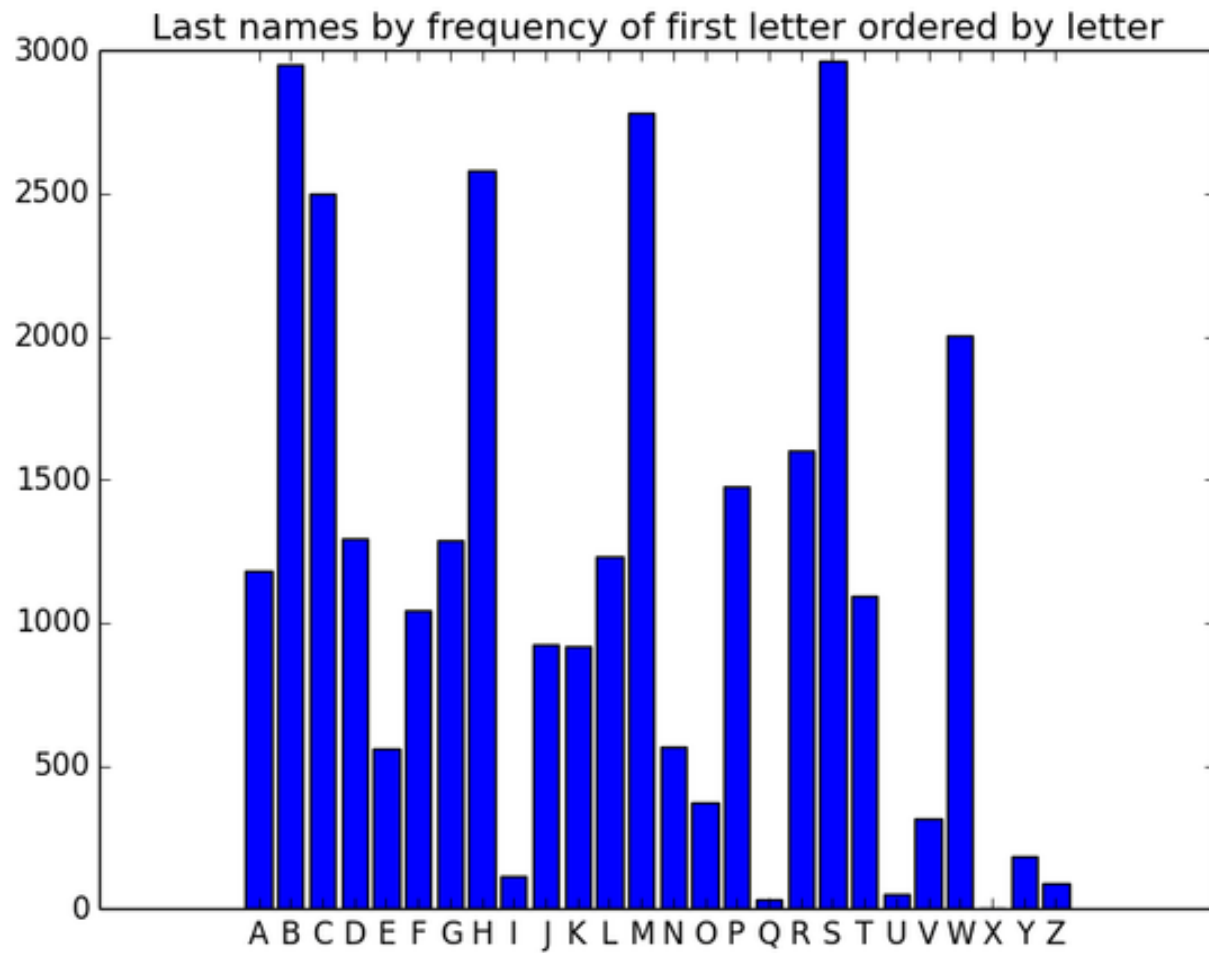
Machines are heterogeneous

Equal partitioning of inputs is hard...

Equal partitioning of intermediates is even harder...

Hash partitioning
Implementation?

Range partitioning
Implementation?



Hash partitioning

Implementation?

Pairs: Pseudo-Code

```
class Mapper {  
  def map(key: Long, value: String) = {  
    for (u <- tokenize(value)) {  
      for (v <- neighbors(u)) {  
        emit((u, v), 1)  
      }  
    }  
  }  
}
```

```
class Reducer {  
  def reduce(key: Pair, values: Iterable[Int]) = {  
    for (value <- values) {  
      sum += value  
    }  
    emit(key, sum)  
  }  
}
```

Pairs: Pseudo-Code

One more thing...

```
class Partitioner {  
  def getPartition(key: Pair, value: Int, numTasks: Int): Int = {  
    return key.left % numTasks  
  }  
}
```

Hash partitioning
Implementation?

Range partitioning
Implementation?

